Name: Arvind Patil
Roll no.: 2401079
Batch: T21

Experiment 3

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Aim: To deploy multi-service applications using Docker Compose.

Theory:

Docker Compose is a tool used to define and manage multi-container Docker applications. Instead of running multiple docker run commands for each container, Docker Compose allows all services to be configured in a single **docker-compose.yml** file. This file specifies the services, images, ports, networks, volumes, and environment variables required for the application.

A multi-service application consists of two or more containers that work together. For example, a web application may require a web server, a database, and a caching service. Docker Compose automatically creates a network that enables these containers to communicate with each other using their service names.

To deploy an application, Docker Compose reads the docker-compose.yml file, builds any required images, creates the necessary containers, configures networking and storage, and starts all services with a single command:

```
docker compose up --build
```

The --build option ensures that the Docker images are rebuilt before the containers are started. Docker Compose also manages the lifecycle of the application, allowing users to stop and remove all services with:

```
docker compose down
```

Advantages of Docker Compose

- Simplifies deployment of multi-container applications.
- Uses a single configuration file for all services.
- Automatically creates a network for container communication.
- Supports persistent storage using volumes.

Name: Arvind Patil
Roll no.: 2401079
Batch: T21

- Makes development, testing, and deployment consistent across different environments.
- Reduces manual configuration and improves maintainability.

Applications

- Web applications with backend and database services.
- Microservices-based applications.
- Development and testing environments.
- Continuous Integration and Continuous Deployment (CI/CD) workflows.

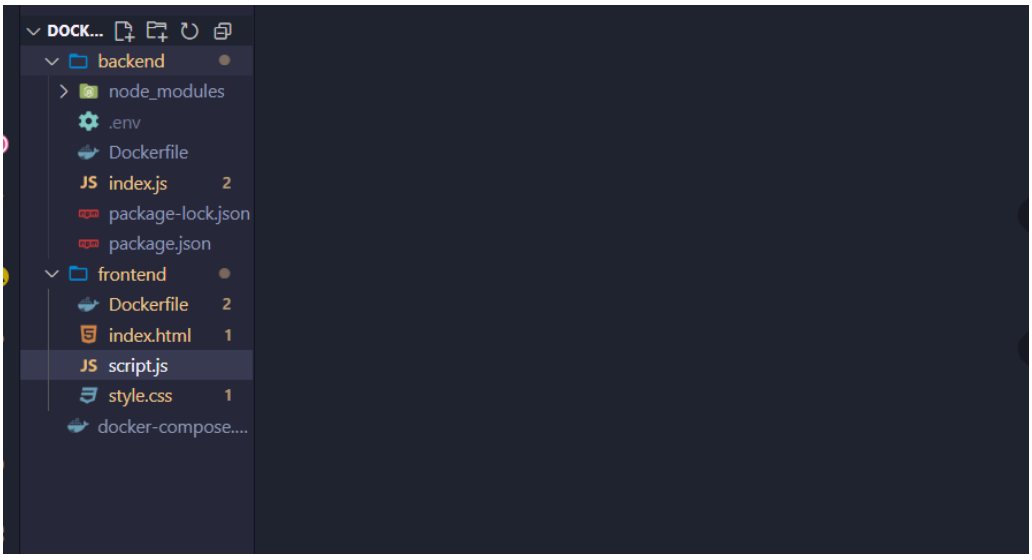
Implementation:

```
C:\Users\15L>docker compose version
Docker Compose version v2.24.6-desktop.1

C:\Users\15L>docker --version
Docker version 25.0.3, build 4debf41
```

Step 1: Create the Project Directory

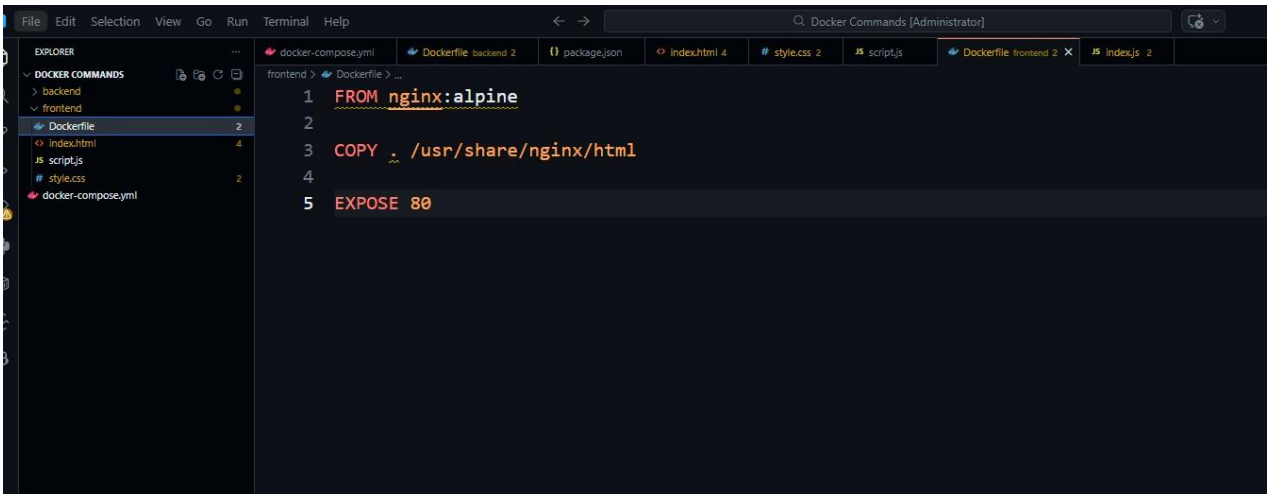
File structure:



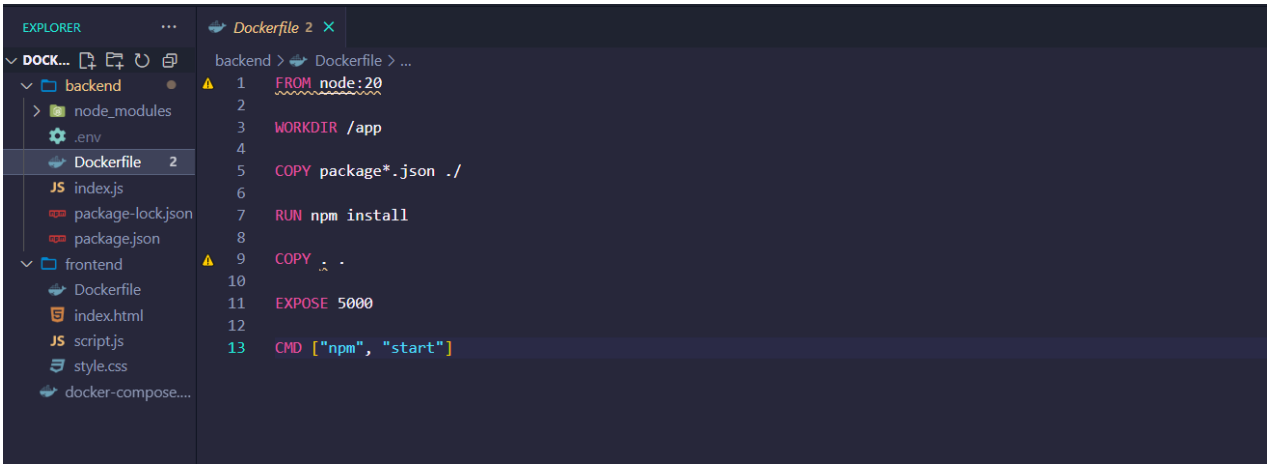
Name: Arvind Patil
Roll no.: 2401079
Batch: T21

Step 2: create docker file

Frontend/Dockerfile:

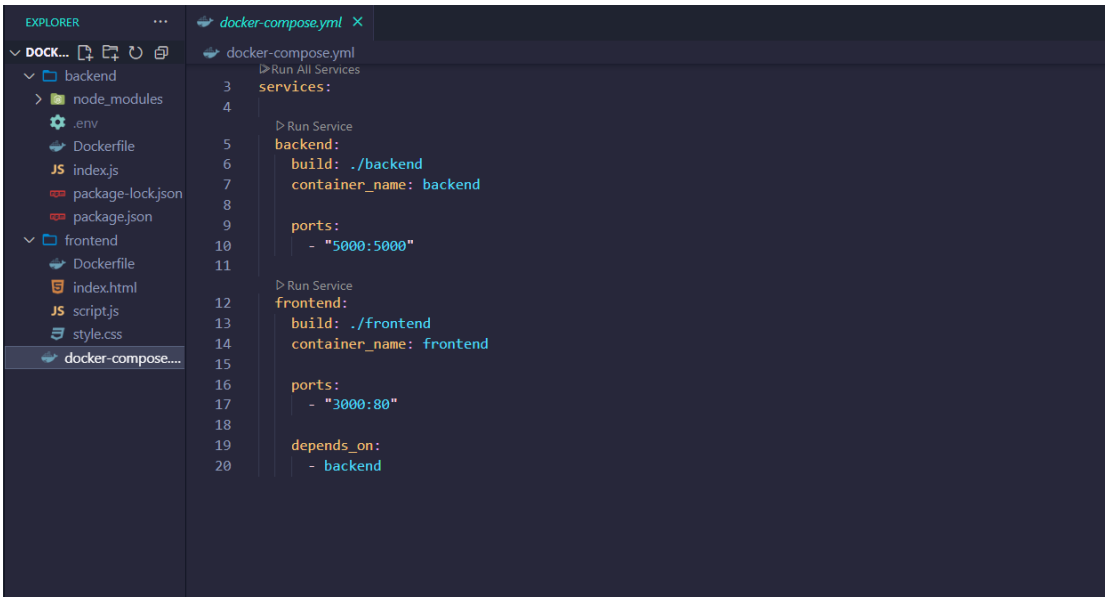


Backend/Dockerfile:

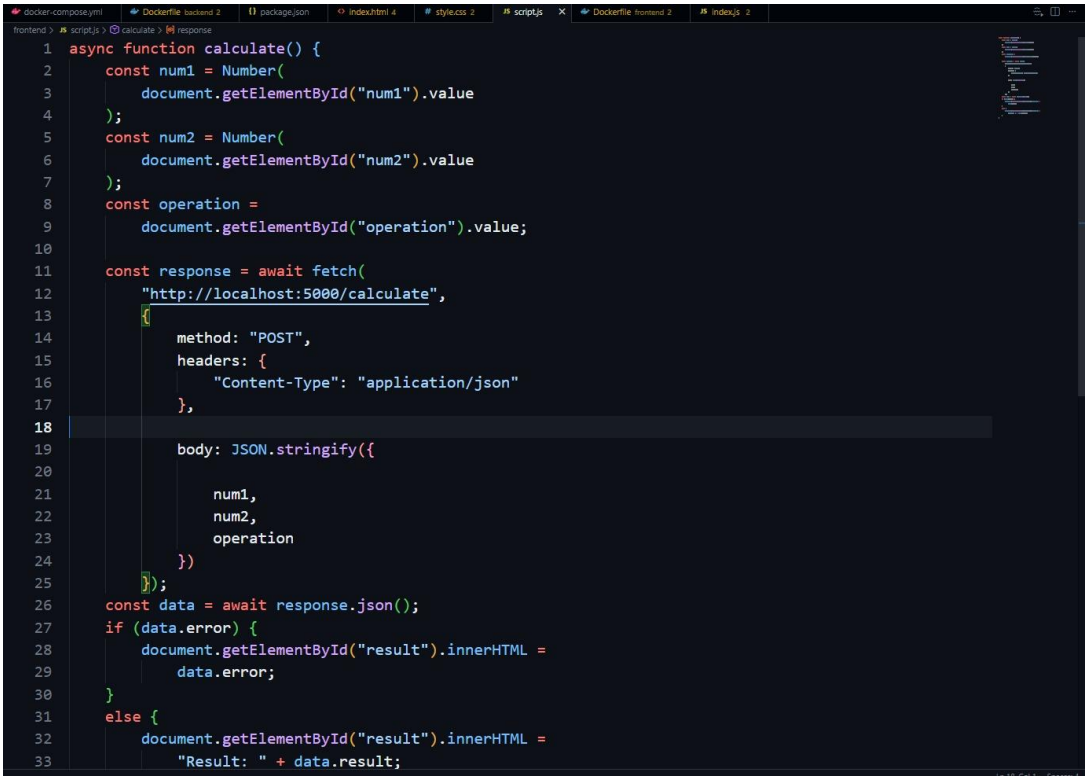


Name: Arvind Patil
Roll no.: 2401079
Batch: T21
Step 3:create docker-compose.yml

Docker-compose.yml:

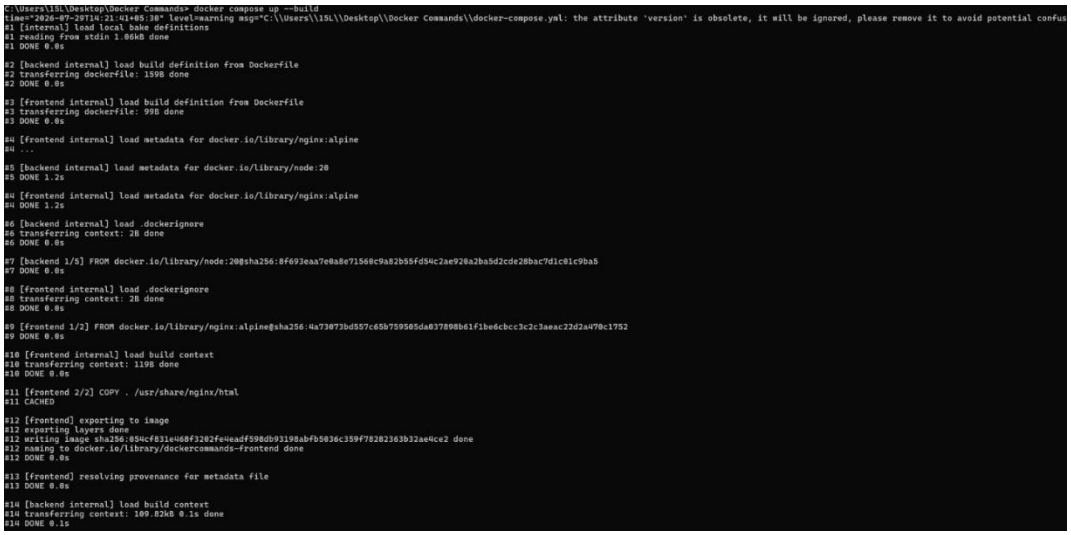


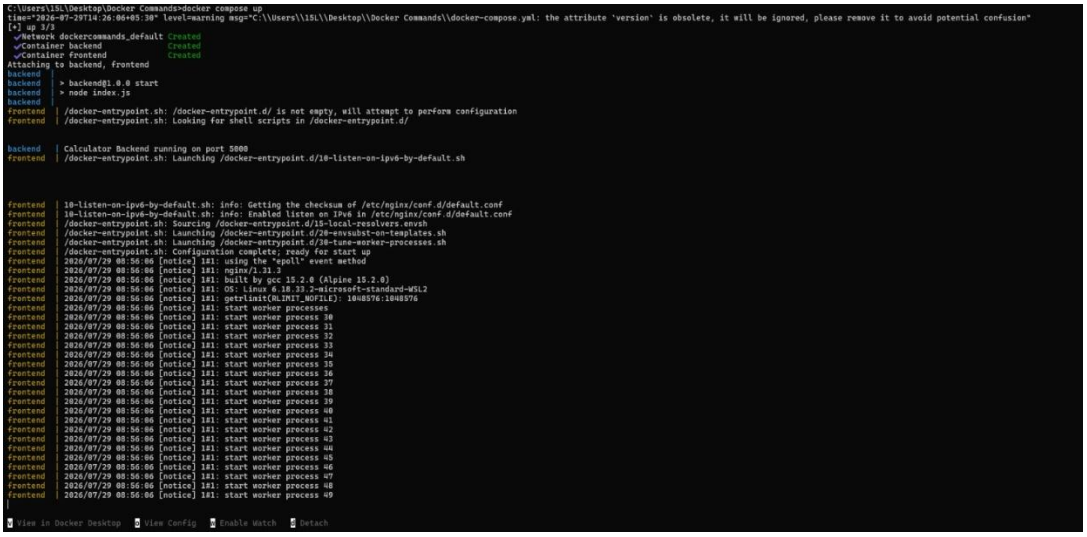
Index.js:



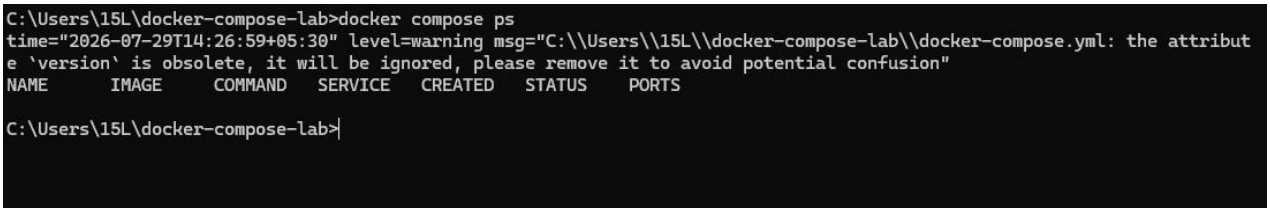
Name: Arvind Patil
Roll no.: 2401079
Batch: T21

Step 4:build and start the services





Step 5: verify the deployment

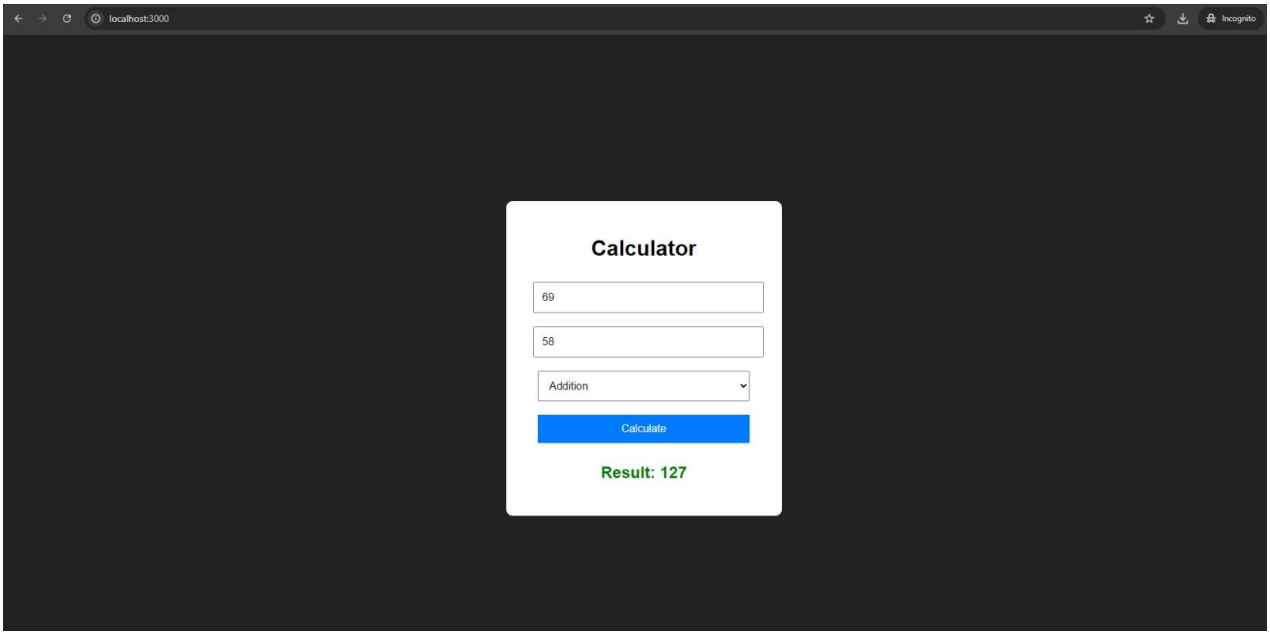


Name: Arvind Patil
Roll no.: 2401079
Batch: T21

Step 6: view application logs

```
PS C:\Users\15L\desktop\093_exp-3> docker compose logs
web-1  |  * Serving Flask app 'app'
web-1  |  * Debug mode: off
web-1  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server i
nstead.
web-1  |  * Running on all addresses (0.0.0.0)
web-1  |  * Running on http://127.0.0.1:5000
web-1  |  * Running on http://172.21.0.2:5000
web-1  | Press CTRL+C to quit
```

Step 7: open project on local host:5000



Step 8: stop and remove services

```
PS C:\Users\15L\desktop\093_exp-3> docker compose down
[+] Running 2/2
 ✔Container 093_exp-3-web-1  Removed                                    10.3s
 ✔Network 093_exp-3_default  Removed                                     0.2s
PS C:\Users\15L\desktop\093_exp-3>
```

Conclusion:

Docker Compose is an efficient tool for deploying and managing multi-service applications using a single configuration file. It automates the process of building images, creating containers, configuring networks, and starting services with minimal effort. By simplifying container orchestration, Docker Compose ensures consistency across development, testing, and deployment environments, making application management easier, faster, and more reliable.